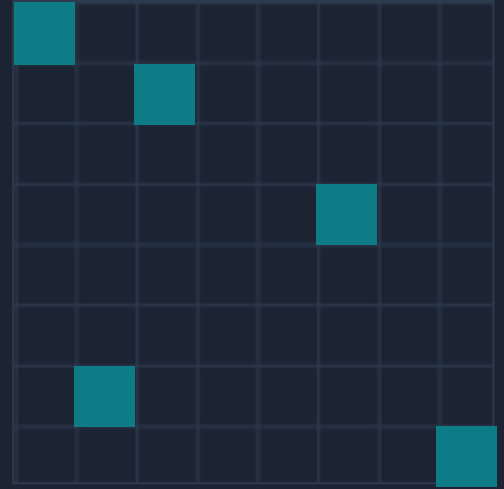


PORTFOLIO

Lim Seongjae

임성재



Email 99dlatjdwo@gmail.com

GitHub github.com/LSJ0621



임성재

Email 99dlatjdwo@gmail.com

GitHub github.com/LSJ0621

문제를 정의하는 개발자 임성재입니다. 요구받은 기능을 그대로 만들기보다 문제를 먼저 정의하고 설계해, 2~3개월 멈춰 있던 논의를 일주일 만에 정리한 경험이 있습니다. 앞으로는 더 다양한 서비스와 사용자 속에서 진짜 문제를 찾아 풀어 보고 싶습니다.

TECH STACK · PRIMARY

LANGUAGES



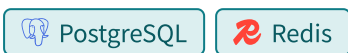
BACKEND



FRONTEND



DATABASE



ORM



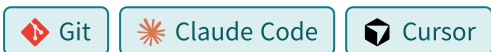
INFRA



TESTING



TOOLS



TECH STACK · EXPERIENCED

BACKEND



FRONTEND



DATABASE



MESSAGING



INFRA



MOBILE



EDUCATION

2018.03 - 2024.08 경희대학교(국제캠퍼스) 우주과학과
복수전공 기계공학과

2024.11 - 2025.05 한화시스템 BEYOND SW CAMP
11기
플레이데이터평생교육원

CAREER

2026.05 - 2026.09 풀링포레스트 · 폴스택 개발자
유통사 · 건설사 ERP 2건 개발

PART 1
p.4 - 5

실무 — 유통사 신규 ERP

- 04 배송 예외 통합 처리 — Before / After · 운영 개선
- 05 예외 처리 AI 에이전트 — 설계 · 프로토타입

PART 2
p.6 - 9

PickEat — AI 메뉴 추천 에이전트

- 06 프로젝트 개요 · 아키텍처 · 기술 스택
- 07 가게추천 — 토큰 13,000 → 1,600 · \$0.043 → \$0.002
- 08 메뉴추천 3단계 파이프라인 — \$0.038 → \$0.012
- 09 Troubleshooting — 문제 3건

PART 3
p.10 - 13

HiddenBar — 위치 기반 로컬 술집 검색

- 10 프로젝트 개요 · 아키텍처 · 기술 스택
- 11 검색 — PostGIS 반경 + pg_trgm 오타 허용
- 12 길안내 + 실시간 위치 — 도보 병합 · 60fps 보간
- 13 Troubleshooting — 문제 3건

p.14

배운 점 / 느낀 점

- 14 PickEat · HiddenBar 회고

PART 4
p.15 - 17

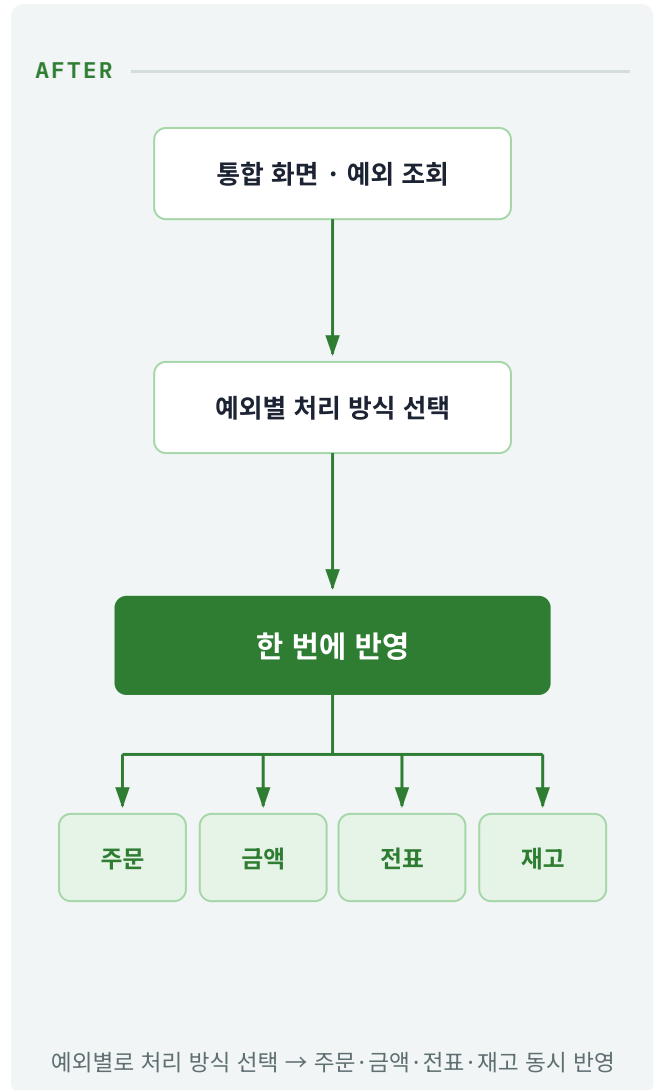
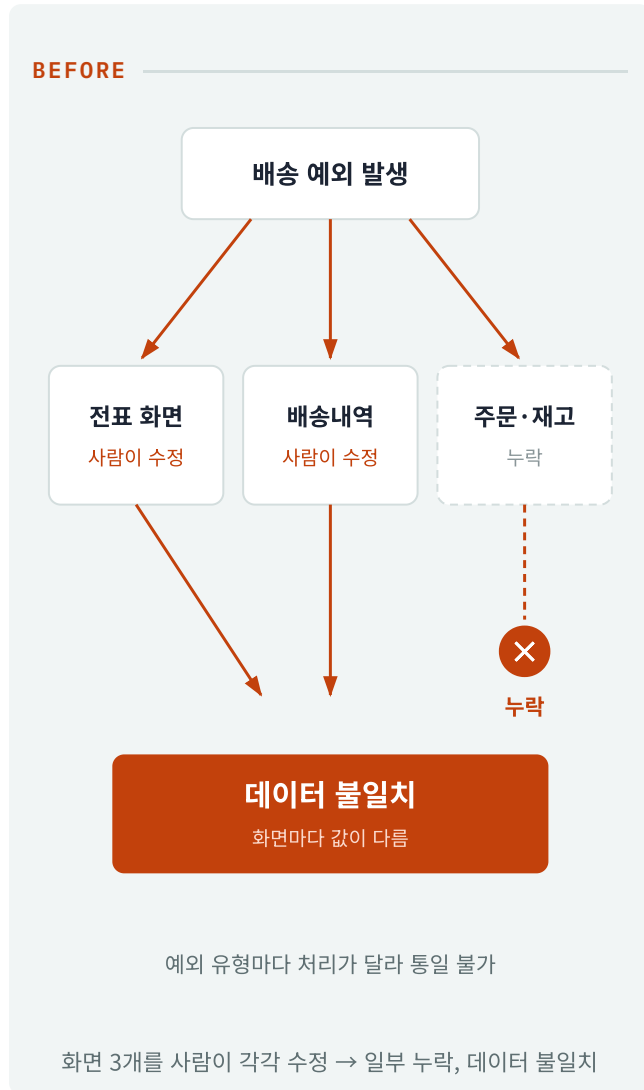
일하는 방식 — AI 개발 운영 체계

- 15 컨텍스트 보존 — 메인 세션은 결정만, 실행은 위임
- 16 지침과 hook의 경계 · 스킬을 만들거나 가져오는 기준
- 17 규칙이 생긴 이유 · 공개 저장소

배송 예외 처리, 여러 화면 수동 수정에서 한 화면 통합 처리로

2026.06 - 2026.07 · 1인 풀스택 · Next.js · TypeScript · Prisma · PostgreSQL · GCP · 신규 ERP 2차 개발

BEFORE / AFTER — 배송 예외 처리 흐름



1화면 · 4개 데이터 동시 반영

예외 통합 처리 결과

운영 개선 — 별도로 처리한 두 건



예외 처리 AI 에이전트, 현업이 쓰던 화면 안에서

2026.07 · 제공 방식과 흐름 설계 · HTML 프로토타입 시연

제공 방식 — 현업이 어떻게 요청하게 할 것인가

MCP · CLI

구현은 쉬움
새 도구를 익혀야 함

ERP 내장 채팅 선택

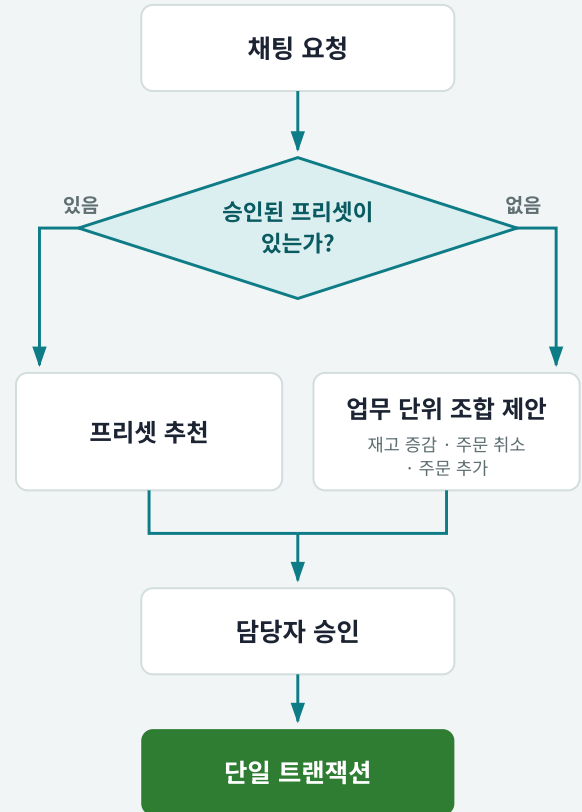
익숙한 화면에서 대화하듯 요청

WHY — 왜 이 설계가 필요한가

고객사가 예외 처리 고도화 방향을 **2~3개월간 정하지 못함**

현업 담당자 연령대가 높아 **새 도구에 거부감**

처리 흐름



중간 실패 시 전체 롤백

프리셋 = 담당자가 승인해 둔 예외 유형별 처리 조합. 있으면 추천하고, 없으면 업무 단위를 조합해 제안

프로토타입 시연

시연
결과

고도화 방향 미확정
2~3개월

HTML 프로토타입 시연
현업 담당자 대상

방향 확정

후속 개발 범위 확정

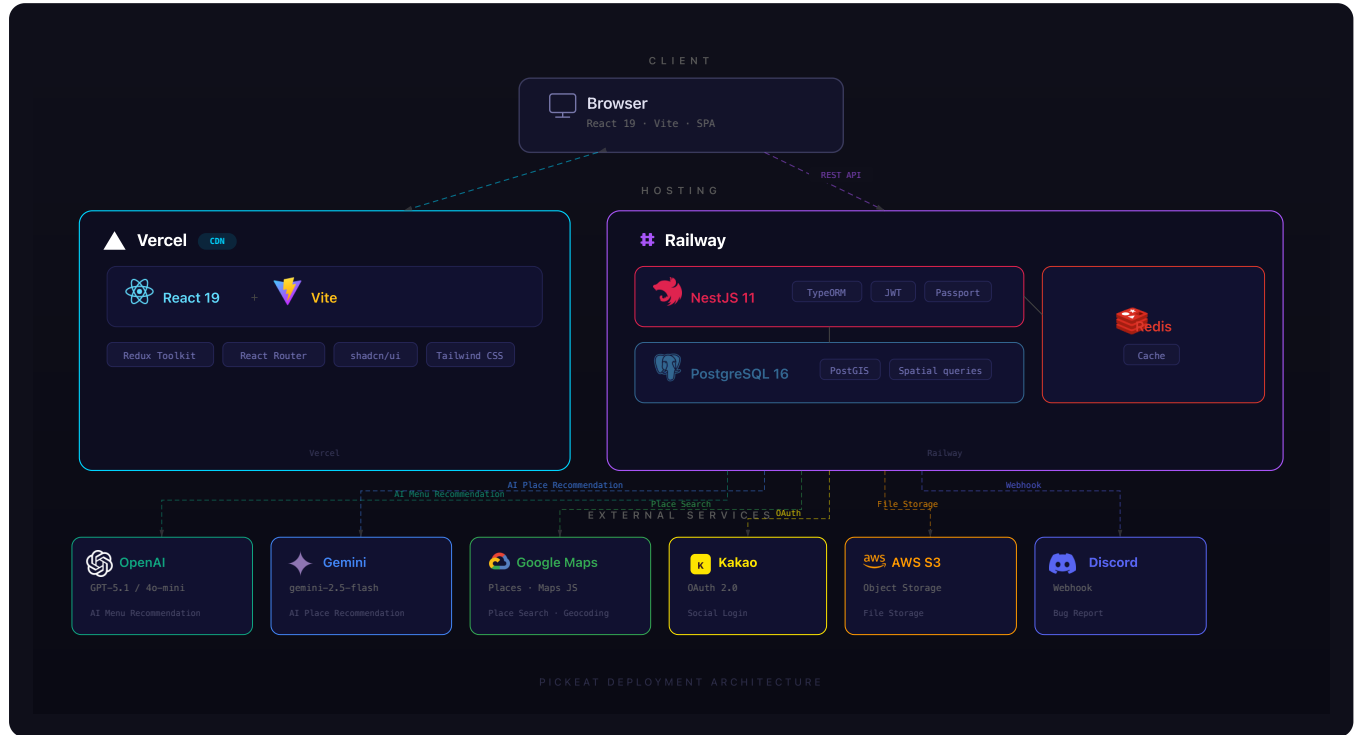
AX는 현업이 받아들일 수 있는 선에서 새로운 기술을 적용할 때 비로소 효과를 낸다

PickEat —

취향·트렌드·가게 리뷰를 종합해 메뉴와 가게를 추천하는 AI 에이전트

2025.11 - 2026.04 · 1인 개발 · 기획 · 설계 · 개발 · 배포 · github.com/LSJ0621/PickEat_BE · [PickEat FE Web](#)

ARCHITECTURE



테스트 Backend 546건 · Frontend 406건 · 총 952건 | 커버리지 Backend 78.6% (Line)

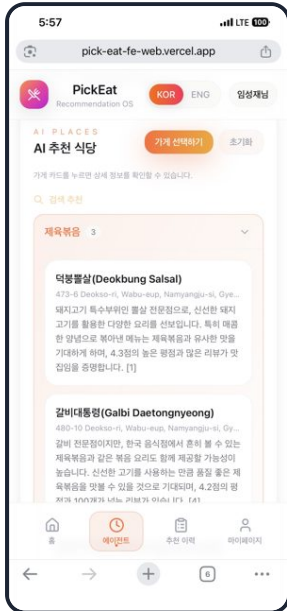
TECH STACK

BACKEND	NestJS	TypeScript	TypeORM
DATABASE	PostgreSQL	PostGIS	Redis
AI	OpenAI API	Google Gemini API	
INTEGRATION	Kakao OAuth	Discord Webhook	
INFRA	Docker	AWS S3	Railway
			Vercel
TESTING	Jest	Playwright	
FRONTEND	React	Tailwind CSS	shadcn/ui

가게추천 — 입력 토큰 약 13,000 → 1,600, 요청당 \$0.043 → \$0.002

유저 좌표 기반 · Gemini Search + Maps Grounding

SCREEN — 화면



추천 결과

WHY — 왜 GEMINI를 선택했는가

기능	OpenAI	Gemini
웹 검색 (Search Grounding)	✓	✓
지도 검색 (Maps Grounding)	✗	✓ Google Maps 연동
웹검색 결과 입력토큰 과금	과금됨	과금 없음

지도 검색 지원 + 검색 결과 토큰 과금 없음 → Gemini

RESULTS — 성과

요청당 비용

-95%

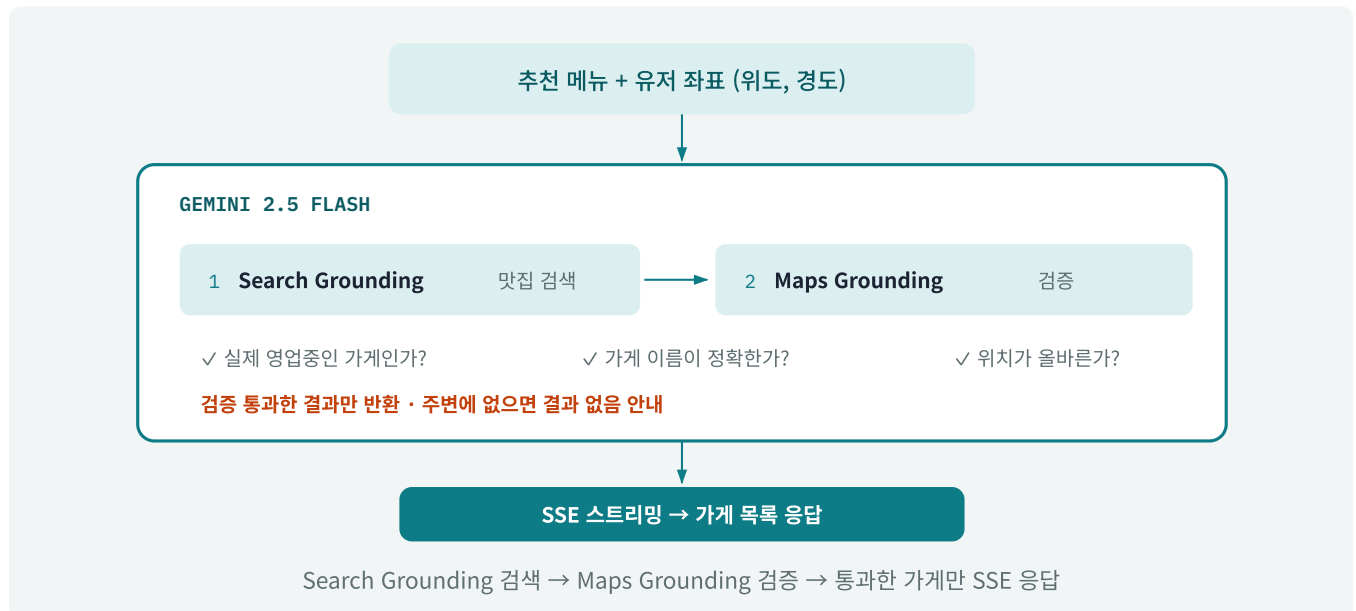
Places 응답 대신 Grounding 정제

검증

통과한 가게만 반환

실존·영업중·위치, 없으면 결과 없음

HOW — 검색 + 검증 플로우

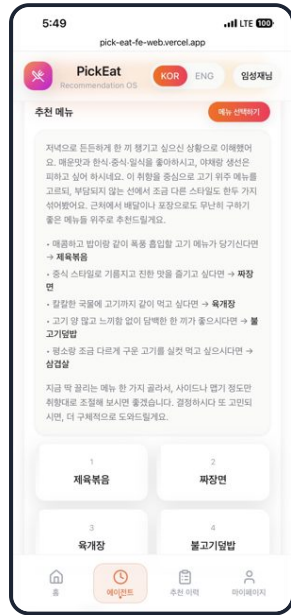


메뉴추천 3단계 파이프라인 — 요청당 비용 \$0.038 → \$0.012

"오늘 저녁 뭐먹지" → 요청 검증 · 트렌드 분석 · 최종 추천 · SSE 스트리밍

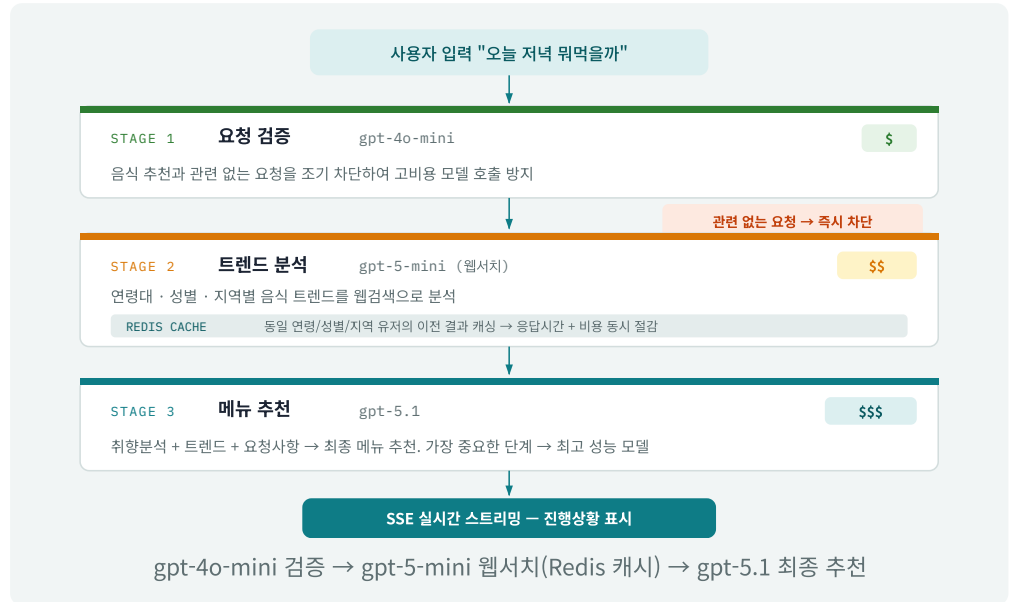
WHY 간단한 요청 검증에도 고비용 모델을 호출하던 구조 → 비싼 모델(gpt-5.1)은 최종 추천에만

SCREEN — 화면



추천 결과

HOW — 3단계 파이프라인



RESULTS — 성과

요청당 비용

-67%

단일 호출 14,000토큰을 3단계로 분산

무관 요청

Stage 1에서 차단

고비용 호출 방지 · Redis 캐싱

취향분석 비용

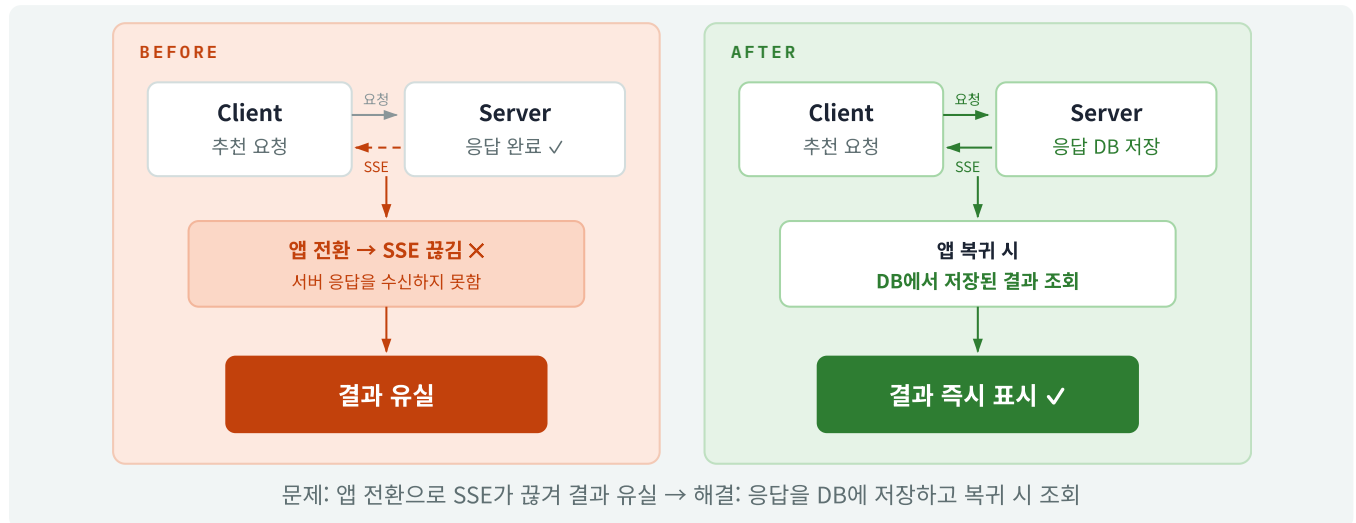
-50%

매일 자정 Batch API 일괄 분석

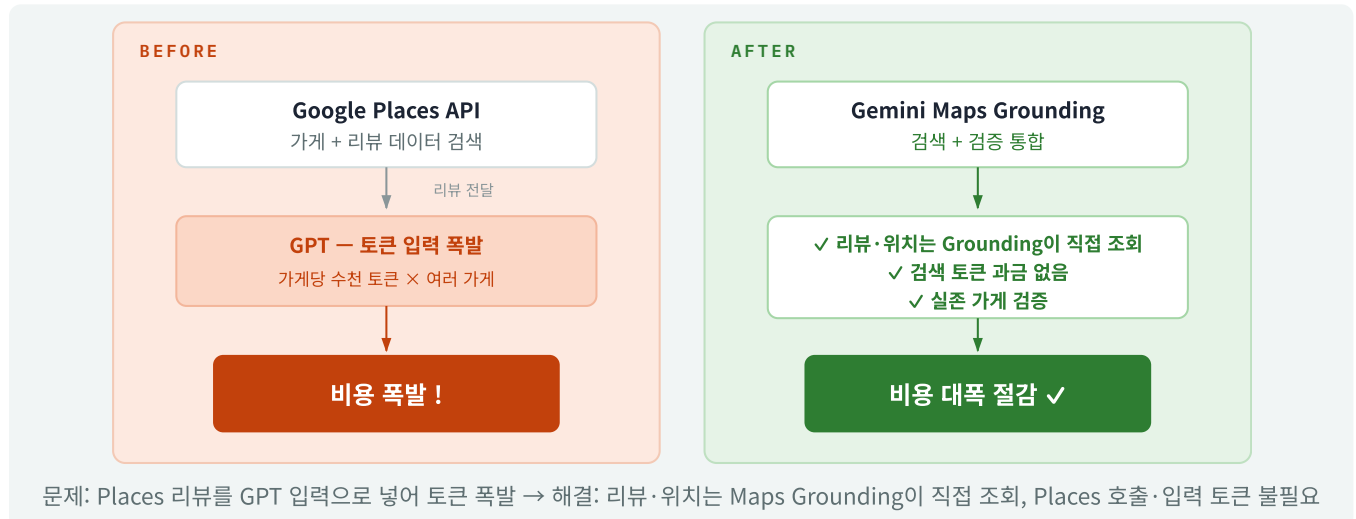
실제로 마주친 문제 3건

문제 한 줄 → 해결 한 줄

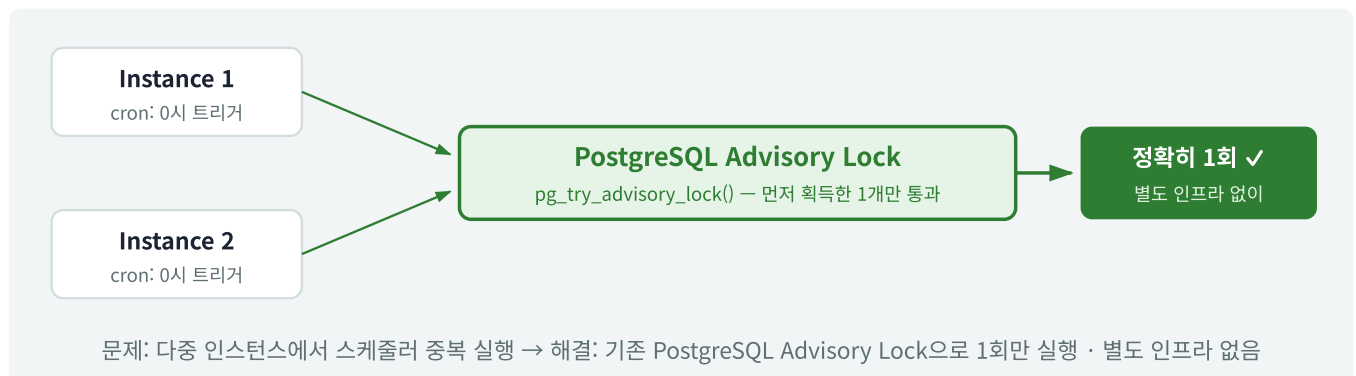
01. 모바일 SSE 끊김 — 추천 결과 유실



02. Google Places API 비용 폭발 — Gemini Maps Grounding 전환



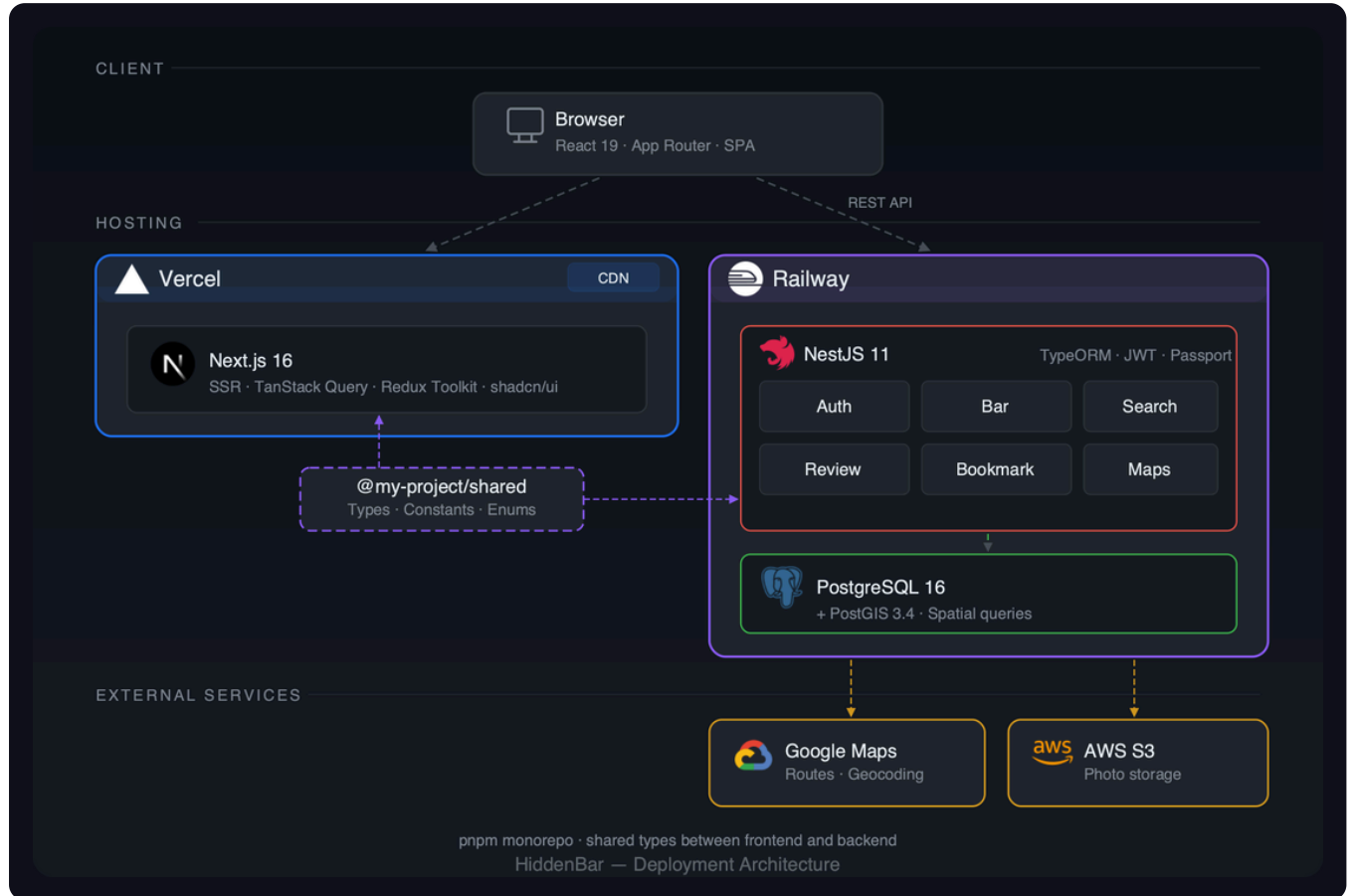
03. 스케일아웃 대비 스케줄러 중복 실행 방지 — PostgreSQL Advisory Lock



HiddenBar — 동남아 로컬 술집을 검색하고 경로를 안내하는 위치 기반 서비스

2026.03 - 2026.04 · 1인 개발 · 기획 · 설계 · 개발 · 배포 · github.com/LSJ0621/HiddenBar

ARCHITECTURE



테스트 Backend 924건 · Frontend 547건 · 총 1,471건 | 커버리지 Backend 96.9% · Frontend 80.5% (Line)

TECH STACK

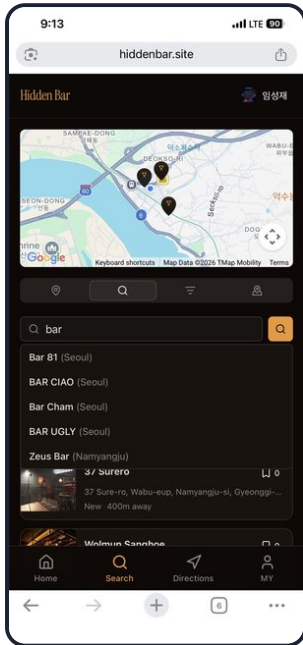
BACKEND	 NestJS	 TypeScript	 TypeORM	
DATABASE	 PostgreSQL	PostGIS	pg_trgm	
FRONTEND	 Next.js	 React	 Tailwind CSS	 shadcn/ui
MAPS	Google Routes API			
TESTING	 Jest	Playwright		
INFRA	 Docker	 Vercel	 Railway	

검색 — PostGIS 반경 검색 + pg_trgm 오타 허용, 검색 모드는 서버가 자동 판별

주소 · 이름 · 복합 · Map Pin 4탭 · GiST / GIN 인덱스 · 자동완성 요청 최소화

WHY LIKE 쿼리로는 반경 검색·오타 허용 불가 → 별도 검색 엔진 없이 PostgreSQL 확장(PostGIS + pg_trgm)으로

SCREEN — 화면



이름 검색 자동완성

RESULTS — 성과

검색 범위

최대 50km

GiST 인덱스 + geography 타입

오타 허용

이름을 몰라도 검색

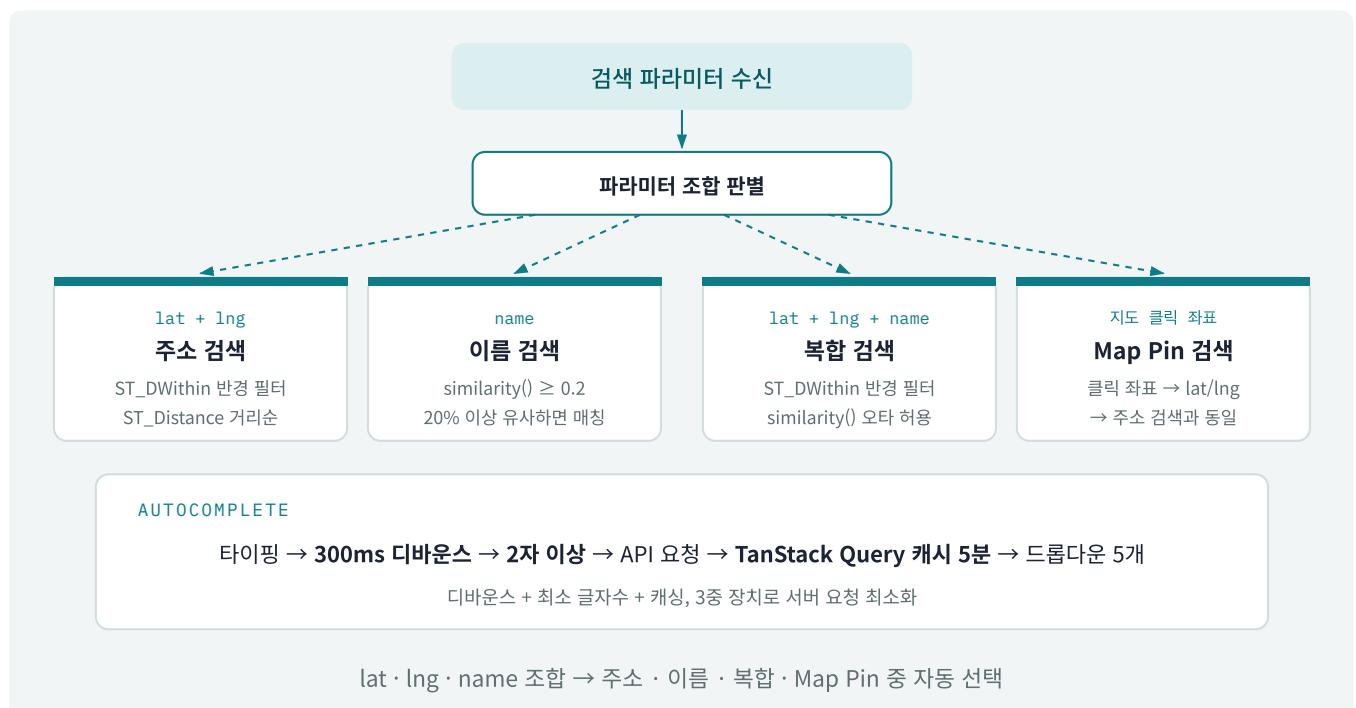
similarity 0.2 기준 유사 매칭

자동완성

서버 요청 최소화

300ms 디바운스 · 2자 이상 · 5분 캐시

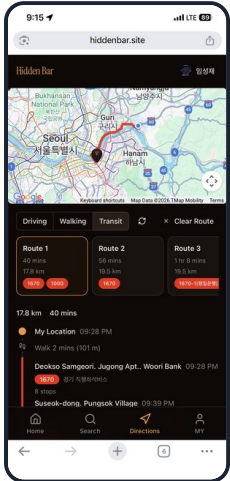
HOW — 검색 모드 자동 판별



길안내 — 연속 도보 구간 병합 실시간 위치 — 서버 없이 60fps 보간

Google Routes API v2 프록시 · Geolocation + Exponential Decay(지수 감쇠) 보간 · GPS 노이즈 필터

길안내 — 경로 요청 ~ 렌더링

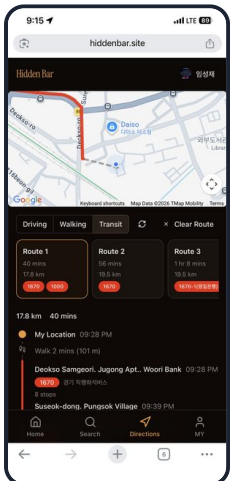


대중교통 타임라인

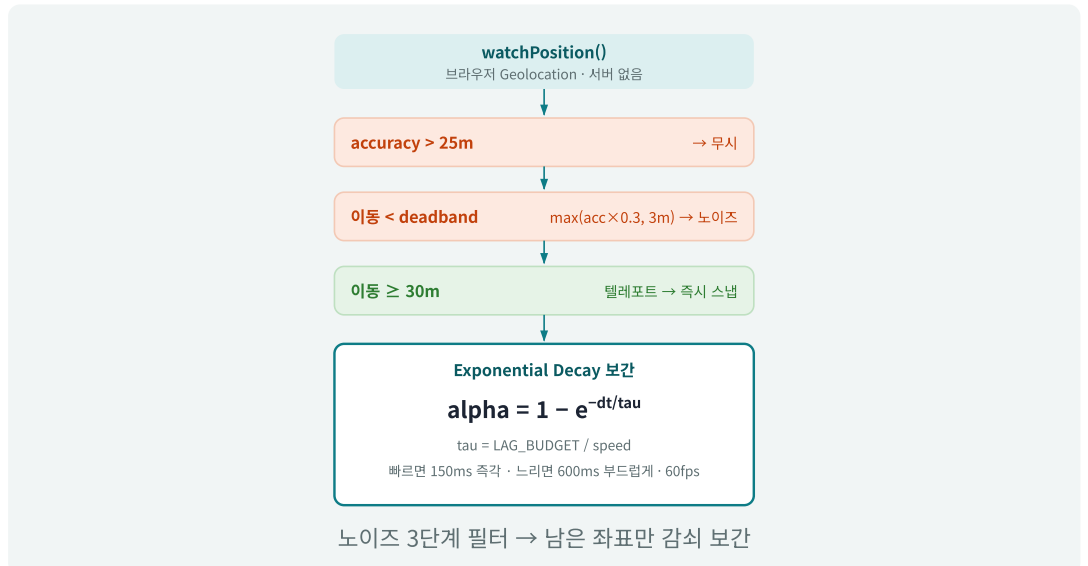


WHY 위치는 서버에 저장하지 않음 — 프라이버시 · 지연 · 서버 비용 → 클라이언트에서만 처리

실시간 위치 — 수신 ~ 마커 렌더링



현재 위치 마커



RESULTS — 성과

프록시

API 키 보호

NestJS 프록시 · Rate Limit 10회/분

모드 전환

즉시 응답

sessionStorage 캐시 · 재호출 없음

위치 서버 비용

0

Geolocation만 · WebSocket 없음

실시간 위치 추적에서 마주친 문제 3건

문제 한 줄 → 해결 한 줄

01. 위치 마커 순간이동 — 좌표 갱신 시 점이 워프

BEFORE

좌표 A · t=0s 좌표 B · t=1s

position prop 직접 전달 → 뚝뚝 끊김

AFTER

현재 목표

Exponential Decay 보간 → 60fps 부드러운 이동

문제: 좌표가 바뀔 때마다 즉시 점프 → 해결: Exponential Decay 보간으로 60fps 이동

02. 도보 → 대중교통 전환 시 마커가 건물 안으로 튕는 현상

BEFORE

도보 5km/h
tau 보행 기준 고정

→

대중교통 40km/h
보간이 못 따라감

뒤쳐진 마커가 건물 안에 표시
위치 왜곡

AFTER

$\tau = \text{LAG_BUDGET} / \text{speed}$

도보 (느림)
tau 600ms 부드럽게

대중교통 (빠름)
tau 150ms 즉각

30m+ 점프 → 보간 스킵, 즉시 스냅 · 위치 왜곡 제거

문제: 속도 급변에 보간이 못 따라감 → 해결: 속도 기반 동적 tau + 30m 이상은 즉시 스냅

03. GPS 오차 — 반대편 도로에 위치 표시

BEFORE

실제 위치
●

GPS 표시
●

도보 A · 차도 · 도보 B

~15m 오차

노이즈가 그대로 반영 → 마커가 좌우로 떨림

AFTER

① 정확도 필터 accuracy > 25m → 무시

② Deadband 필터 이동 < max(acc×0.3, 3m) → 노이즈

③ 스냅 임계값 0.5m 이내 → 스냅 후 중지

GPS 떨림 최소화

문제: GPS 정확도 한계(10~20m)가 마커에 그대로 반영 → 해결: 정확도 · Deadband · 스냅 3단계 필터

배운 점 / 느낀 점

PickEat과 HiddenBar를 끝내고 남은 것

PICKEAT

1. 신중한 프레임워크 선택의 중요성

- LangChain/LangGraph 없이 프롬프트 하나로 제어 가능하다고 판단
- 취향 정보가 컨텍스트에 포함되고, hallucination을 줄이기 위한 규약까지 추가하다 보니 컨텍스트가 길어져 오히려 hallucination이 증가
- 단계 분리 + 모델 분산으로 해결했으나, 모델마다 요청 양식이 다르고 단계별 분기처리도 필요해 코드 복잡도 급증
- 처음부터 오케스트레이션 프레임워크를 도입해 단계 관리와 실패 시 복구 흐름까지 체계적으로 설계하는 것이 더 나았음

→ 프레임워크 선정 시 더 신중해야 함을 배움

2. TDD를 처음부터 적용해야 하는 이유

- 기능이 늘어날수록 새 기능이 기존 동작에 미치는 영향 파악이 어려움
- 테스트를 충분히 갖춰 두었다면 영향 범위를 훨씬 빠르게 확인 가능

→ 프로젝트 중반에 TDD를 알게 되어 적용하지 못한 것이 아쉬웠고, 다음 프로젝트에서는 처음부터 적용하겠다고 결심

3. 공식 문서를 꼼꼼히 읽는 습관

- Google Places API 비용 절감을 위해 Redis 캐싱 도입
- 이후 공식 문서 재확인 시 응답 데이터 캐싱이 약관 위반임을 뒤늦게 발견, 구조 전면 수정

→ 기능 구현 전 공식 문서의 이용 약관과 제약 사항까지 꼼꼼히 읽는 습관이 불필요한 재작업을 줄여준다는 것을 배움

HIDDENBAR

1. TDD와 문서화를 습관으로

- 처음부터 적용했지만 일정에 쫓겨 중반부터 서비스 코드를 먼저 작성. 테스트와 문서화가 부채로 쌓임
- 그래도 먼저 작성한 테스트와 검증 단계 덕에 이전보다 검증이 빨랐음

→ 완벽하진 않지만 방향은 맞다

2. 있는 기술로 충분한지 먼저 본다

- 검색엔 Elasticsearch가 필수라고 생각했으나, PostGIS GiST와 pg_trgm GIN 인덱스로 별도 인프라 없이 충분했음

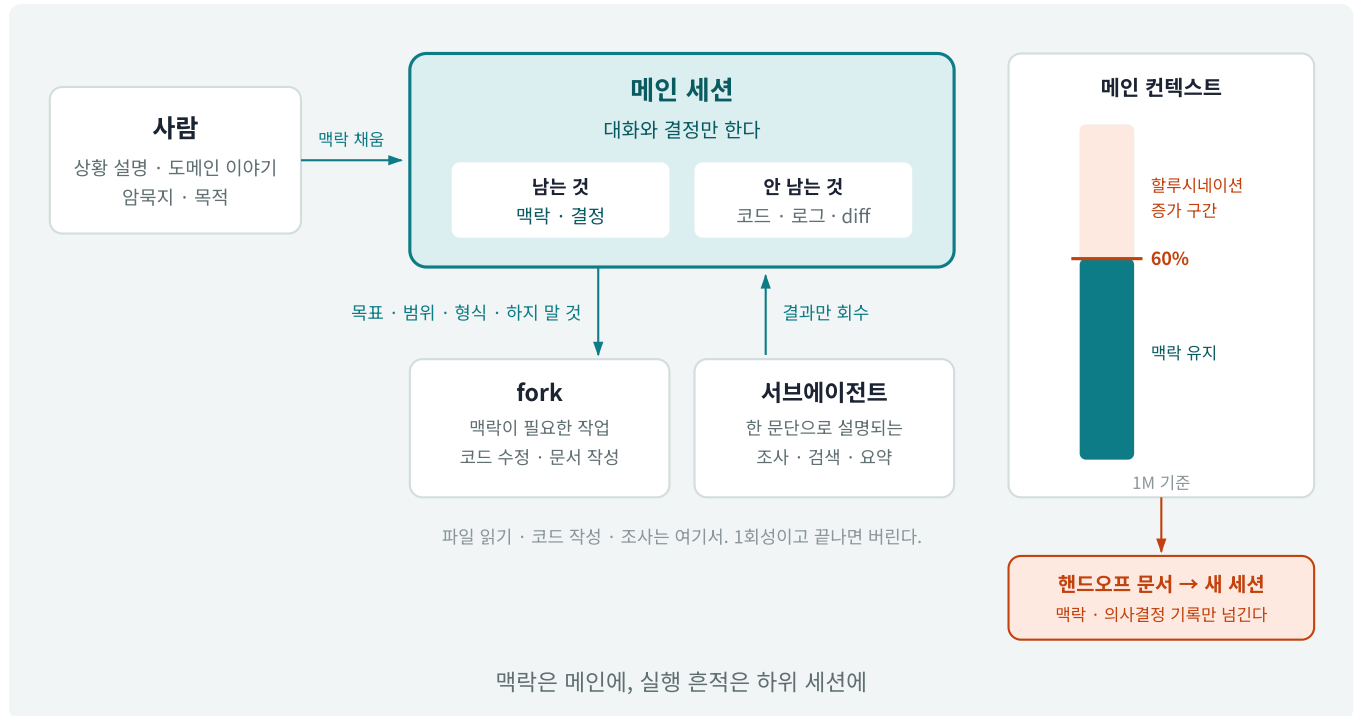
→ 새 기술 도입 전에 쓰고 있는 기술의 가능성을 먼저 탐색

컨텍스트 보존 —

메인 세션은 결정만, 실행은 서브에이전트에 위임

CLAUDE.md · 세션 운영 규칙

HOW — 세션 구조



우선순위 — 규칙이 충돌할 때 고르는 순서



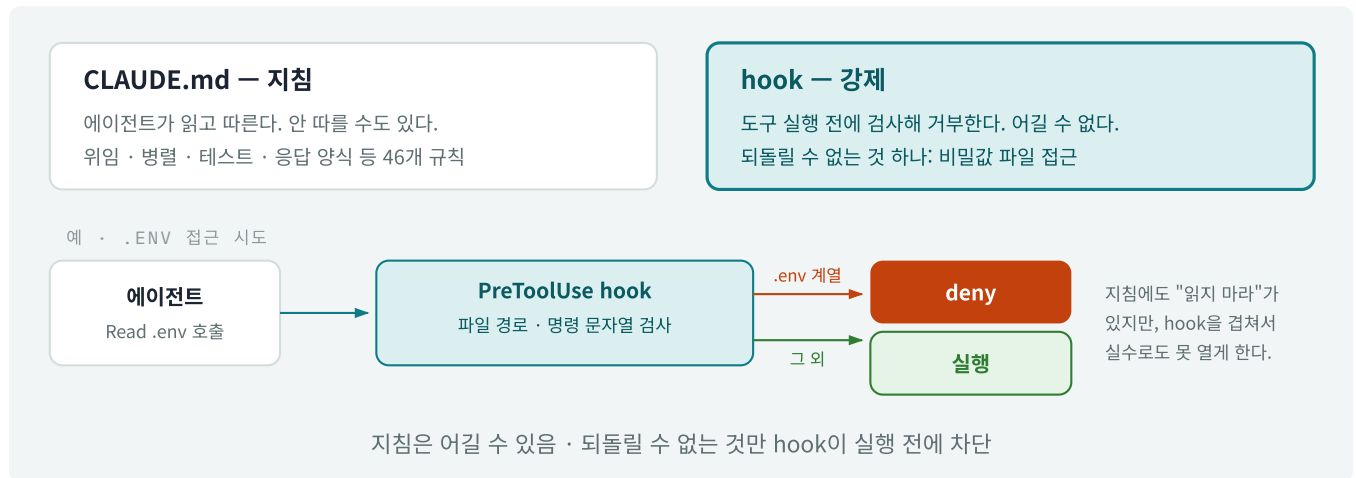
핵심

- 맥락은 사람이 채움 · 작업 전 상황 설명 먼저
- 60% 초과 시 정확도 하락 → 실행은 위임, 결과만 회수
- 그래도 넘치면 맥락·결정만 핸드오프 문서로 새 세션에 인계

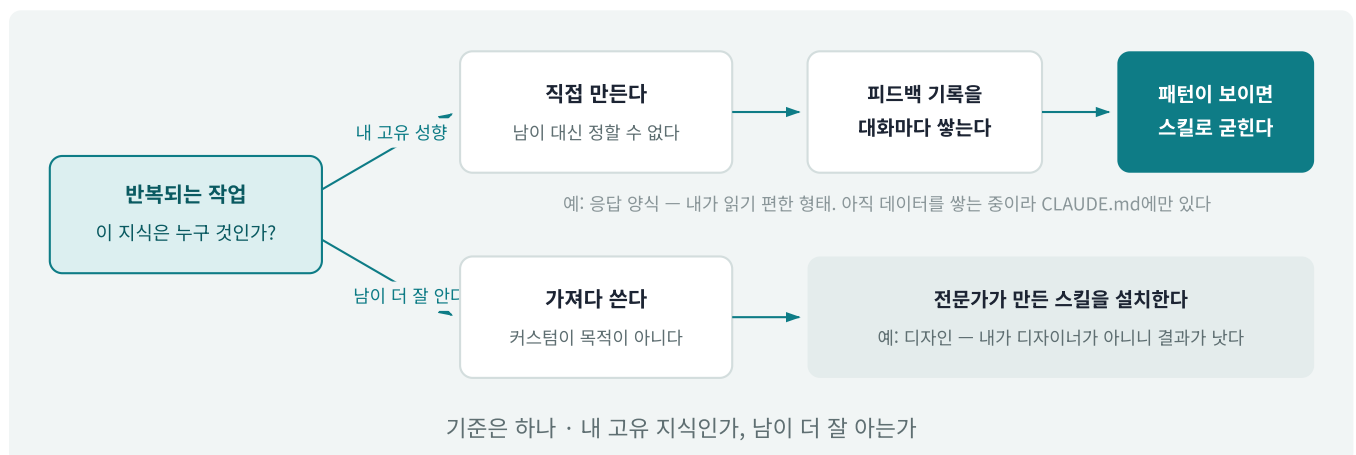
비밀값 파일 접근은 규칙이 아니라 hook으로 차단

지침(CLAUDE.md) · hook · 스킬

A — 지침(CLAUDE.MD)과 HOOK은 다르다



B — 스킬을 만들거나 가져오는 기준



규칙이 생긴 이유

CLAUDE.md 규칙 5개와 그 이유

RULES — 규칙 5개

규칙

메인 세션은 결정만. 읽기 · 작성 · 조사는 위임

이유

맥락이 코드·로그에 밀려남 · 60% 초과 시 할루시네이션

규칙

브리핑은 목표 · 범위 · 형식 · 하지 말 것

이유

하나라도 빠지면 결과가 어긋남

규칙

도메인 로직 · 버그 수정은 테스트 먼저

이유

다른 기능이 망가지지 않게 · 회귀 방지

규칙

응답은 핵심만, 짧게. 파일명은 요청할 때만

이유

이해가 어려우면 판단이 늦어짐

규칙

비밀값은 출력하지 않는다

이유

지침만으로 못 막아 hook을 겹침

공개 저장소

CLAUDE.md 원문 · 비밀값 차단 hook 발체 · 이유를 정리한 README



github.com/LSJ0621/open_claude_setting

THANK YOU

읽어주셔서 감사합니다.

AI를 제품에 넣는 일과 AI로 개발하는 일, 둘 다 같은 원칙으로 합니다. 왜 그렇게 했는지를 남기고, 어기면 안 되는 것은 장치로 막습니다.

Name 임성재
Email 99dlatjdwo@gmail.com
GitHub github.com/LSJ0621